You want to build more interactive features now that you have the basics covered. Users are used to Facebook and Twitter and want to be notified whenever their name is mentioned in a team conversation so they can chime in.

You decide to build a `Mentions` component that parses every message written and adds the mentioned contacts to a notification table, which is polled from the web application to notify mentioned users. The web application polls other information more often to reflect changes to users more quickly because you want to give them a truly interactive experience.

You don’t want to slow down the conversations by adding database code directly to the application, so you add a message queue. Every message written is sent to this queue asynchronously, and a separate process receives messages from the queue, looks up the users, and writes a record in a notifications table.

The database is getting hammered at this point. You find out that the automated polling of the database and the `Mentions` component are causing database performance problems. You separate the `Mentions` component as a service and give it its own database, which contains the notifications table and a copy of the users table; these tables are kept up to date with a database synchronization job, as shown in figure 1.7.